Week 9: Sales and Customer Analytics System

To analyze sales and customer data using SQL aggregate functions and generate business analytics reports to identify customer purchasing behavior, product performance, total sales, and revenue generated by each category.

1. Apply SQL Aggregate Functions

Aggregate functions such as COUNT(), SUM(), AVG(), MAX(), and MIN() are used to calculate and summarize data from multiple records.

1. Total number of orders placed

```
SQL> SELECT COUNT(*) AS Number_Of_Orders_Placed
  2  FROM Orders;

NUMBER_OF_ORDERS_PLACED
-----------------------
                      5
```

2. Total revenue generated from sales

```
SQL> SELECT SUM(TOTAL_AMOUNT) AS Total_Revenue
  2  FROM Orders;

TOTAL_REVENUE
-------------
        19700
```

For Average Order Value:

```
SQL> SELECT AVG(Total_Amount) AS Average_Order_Value
  2  FROM Orders;

AVERAGE_ORDER_VALUE
-------------------
               3940
```

Highest Order Amount and Lowest Order Amount

```
SQL> SELECT MAX(TOTAL_AMOUNT) AS Highest_Order_Amount
  2  FROM Orders;

HIGHEST_ORDER_AMOUNT
--------------------
                7500

SQL> SELECT MIN(TOTAL_AMOUNT) AS Lowest_Order_Amount
  2  FROM Orders;

LOWEST_ORDER_AMOUNT
-------------------
               1500
```

5. Total Sales for a Specific Period

```
SQL> SELECT SUM(TOTAL_AMOUNT) AS TOTAL_REVENUE
  2  FROM Orders
  3  WHERE ORDER_DATE BETWEEN
  4  TO_DATE('2026-10-01','YYYY-MM-DD')
  5  AND TO_DATE('2026-10-05','YYYY-MM-DD');

TOTAL_REVENUE
-------------
        19700
```

3. Customer Purchase Analysis

1. Total number of orders placed by each customer

```
SQL> SELECT C.CUSTOMER_ID,
  2         C.CUSTOMER_NAME,
  3         COUNT(O.ORDER_ID) AS TOTAL_ORDERS
  4  FROM CUSTOMER C
  5  LEFT JOIN ORDERS O
  6  ON C.CUSTOMER_ID = O.CUSTOMER_ID
  7  GROUP BY C.CUSTOMER_ID, C.CUSTOMER_NAME
  8  ORDER BY TOTAL_ORDERS DESC;

CUSTOMER_ID CUSTOMER_NAME   TOTAL_ORDERS
----------- --------------- ------------
          3 Priya                      1
          6 Rahul                      1
          5 Divya                      1
          4 Karthik                    1
          2 Arun                       1
```

2. Total spending by each customer

```
SQL> SELECT C.CUSTOMER_ID,
  2         C.CUSTOMER_NAME,
  3         SUM(O.TOTAL_AMOUNT) AS TOTAL_SPENT
  4  FROM CUSTOMER C
  5  LEFT JOIN ORDERS O
  6  ON C.CUSTOMER_ID = O.CUSTOMER_ID
  7  GROUP BY C.CUSTOMER_ID, C.CUSTOMER_NAME
  8  ORDER BY TOTAL_SPENT DESC;

CUSTOMER_ID CUSTOMER_NAME   TOTAL_SPENT
----------- --------------- -----------
          5 Divya                  7500
          3 Priya                  4500
          4 Karthik                3400
          2 Arun                   2800
          6 Rahul                  1500
```

3. Average order value for each customer

```
SQL> SELECT C.CUSTOMER_ID,
  2         C.CUSTOMER_NAME,
  3         AVG(O.TOTAL_AMOUNT) AS AVERAGE_SPENDING
  4  FROM CUSTOMER C
  5  LEFT JOIN ORDERS O
  6  ON C.CUSTOMER_ID = O.CUSTOMER_ID
  7  GROUP BY C.CUSTOMER_ID, C.CUSTOMER_NAME
  8  ORDER BY AVERAGE_SPENDING DESC;

CUSTOMER_ID CUSTOMER_NAME   AVERAGE_SPENDING
----------- --------------- ----------------
          5 Divya                       7500
          3 Priya                       4500
          4 Karthik                     3400
          2 Arun                        2800
          6 Rahul                       1500
```

4. Customer with Maximum Purchase Amount

```
SQL> SELECT *
  2  FROM (
  3      SELECT C.CUSTOMER_ID,
  4             C.CUSTOMER_NAME,
  5             SUM(O.TOTAL_AMOUNT) AS TOTAL_PURCHASE_AMOUNT
  6      FROM CUSTOMER C
  7      JOIN ORDERS O
  8      ON C.CUSTOMER_ID = O.CUSTOMER_ID
  9      GROUP BY C.CUSTOMER_ID, C.CUSTOMER_NAME
 10      ORDER BY TOTAL_PURCHASE_AMOUNT DESC
 11  )
 12  WHERE ROWNUM = 1;

CUSTOMER_ID CUSTOMER_NAME   TOTAL_PURCHASE_AMOUNT
----------- --------------- --------------------
          5 Divya                           7500
```

5. Customer with Fewer Purchases

```
SQL> SELECT C.CUSTOMER_ID,
  2         C.CUSTOMER_NAME,
  3         COUNT(O.ORDER_ID) AS TOTAL_PURCHASES
  4  FROM CUSTOMER C
  5  LEFT JOIN ORDERS O
  6  ON C.CUSTOMER_ID = O.CUSTOMER_ID
  7  GROUP BY C.CUSTOMER_ID, C.CUSTOMER_NAME
  8  HAVING COUNT(O.ORDER_ID) = (
  9      SELECT MIN(COUNT(*))
 10      FROM ORDERS
 11      GROUP BY CUSTOMER_ID
 12  );

CUSTOMER_ID CUSTOMER_NAME   TOTAL_PURCHASES
----------- --------------- ---------------
          3 Priya                         1
          6 Rahul                         1
          2 Arun                          1
          4 Karthik                       1
          5 Divya                         1
```

4. Find Top Customers Based on Purchase Amount

1. Top 5 customers based on total spending

```
SQL> SELECT *
  2  FROM (
  3      SELECT C.CUSTOMER_ID,
  4             C.CUSTOMER_NAME,
  5             SUM(O.TOTAL_AMOUNT) AS TOTAL_SPENDING
  6      FROM CUSTOMER C
  7      JOIN ORDERS O
  8      ON C.CUSTOMER_ID = O.CUSTOMER_ID
  9      GROUP BY C.CUSTOMER_ID, C.CUSTOMER_NAME
 10      ORDER BY TOTAL_SPENDING DESC
 11  )
 12  WHERE ROWNUM <= 5;

CUSTOMER_ID CUSTOMER_NAME   TOTAL_SPENDING
----------- --------------- --------------
          5 Divya                     7500
          3 Priya                     4500
          4 Karthik                   3400
          2 Arun                      2800
          6 Rahul                     1500
```

Customer with Maximum Number of Orders

```
SQL> SELECT C.CUSTOMER_ID,
  2         C.CUSTOMER_NAME,
  3         COUNT(O.ORDER_ID) AS TOTAL_ORDERS
  4  FROM CUSTOMER C
  5  JOIN ORDERS O
  6  ON C.CUSTOMER_ID = O.CUSTOMER_ID
  7  GROUP BY C.CUSTOMER_ID, C.CUSTOMER_NAME
  8  HAVING COUNT(O.ORDER_ID) = (
  9      SELECT MAX(COUNT(*))
 10      FROM ORDERS
 11      GROUP BY CUSTOMER_ID
 12  );

CUSTOMER_ID CUSTOMER_NAME   TOTAL_ORDERS
----------- --------------- ------------
          3 Priya                      1
          6 Rahul                      1
          2 Arun                       1
          4 Karthik                    1
          5 Divya                      1
```

Frequent Orders by Each Customer

```
SQL> SELECT C.CUSTOMER_ID,
  2         C.CUSTOMER_NAME,
  3         COUNT(O.ORDER_ID) AS TOTAL_ORDERS
  4  FROM CUSTOMER C
  5  JOIN ORDERS O
  6  ON C.CUSTOMER_ID = O.CUSTOMER_ID
  7  GROUP BY C.CUSTOMER_ID, C.CUSTOMER_NAME
  8  HAVING COUNT(O.ORDER_ID) > 1
  9  ORDER BY TOTAL_ORDERS DESC;

no rows selected
```

High Value Customer

```
SQL> SELECT C.CUSTOMER_ID,
  2         C.CUSTOMER_NAME,
  3         SUM(O.TOTAL_AMOUNT) AS TOTAL_SPENDING
  4  FROM CUSTOMER C
  5  JOIN ORDERS O
  6  ON C.CUSTOMER_ID = O.CUSTOMER_ID
  7  GROUP BY C.CUSTOMER_ID, C.CUSTOMER_NAME
  8  HAVING SUM(O.TOTAL_AMOUNT) >= 5000
  9  ORDER BY TOTAL_SPENDING DESC;

CUSTOMER_ID CUSTOMER_NAME   TOTAL_SPENDING
----------- --------------- --------------
          5 Divya                    7500
```

5. Identify Best-Selling Products

1. Products with Maximum Sales Quantity

```
SQL> SELECT P.PRODUCT_ID,
  2         P.PRODUCT_NAME,
  3         SUM(OD.QUANTITY) AS TOTAL_QUANTITY_SOLD
  4  FROM PRODUCT P
  5  JOIN ORDER_DETAILS OD
  6  ON P.PRODUCT_ID = OD.PRODUCT_ID
  7  GROUP BY P.PRODUCT_ID, P.PRODUCT_NAME
  8  ORDER BY TOTAL_QUANTITY_SOLD DESC;

PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------------------------------
--
TOTAL_QUANTITY_SOLD
-------------------
P004
Headphones
                  3

P001
Laptop
                  1

PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------------------------------
--
TOTAL_QUANTITY_SOLD
-------------------
```

```
PRODUCT_ID
----------
PRODUCT_NAME
----------------------------------------
--
TOTAL_QUANTITY_SOLD
-------------------

P005
Bag
                  1

P002
Mobile

PRODUCT_ID
----------
PRODUCT_NAME
----------------------------------------
--
TOTAL_QUANTITY_SOLD
-------------------
                  1

P003
Watch
                  1
```

2. Products Generating Highest Revenue

```
SQL> SELECT P.PRODUCT_ID,
  2         P.PRODUCT_NAME,
  3         SUM(P.PRICE * OD.QUANTITY) AS TOTAL_REVENUE
  4  FROM PRODUCT P
  5  JOIN ORDER_DETAILS OD
  6  ON P.PRODUCT_ID = OD.PRODUCT_ID
  7  GROUP BY P.PRODUCT_ID, P.PRODUCT_NAME
  8  ORDER BY TOTAL_REVENUE DESC;

PRODUCT_ID   PRODUCT_NAME         TOTAL_REVENUE
------------ -------------------- -------------
P004         Headphones                 7500.00
P002         Mobile                     4500.00
P003         Watch                      3400.00
P001         Laptop                     2500.00
P005         Bag                        1500.00
```

3. Least Selling Products

```
SQL> SELECT P.PRODUCT_ID,
  2         P.PRODUCT_NAME,
  3         SUM(OD.QUANTITY) AS TOTAL_QUANTITY_SOLD
  4  FROM PRODUCT P
  5  JOIN ORDER_DETAILS OD
  6  ON P.PRODUCT_ID = OD.PRODUCT_ID
  7  GROUP BY P.PRODUCT_ID, P.PRODUCT_NAME
  8  ORDER BY TOTAL_QUANTITY_SOLD ASC;

PRODUCT_ID   PRODUCT_NAME         TOTAL_QUANTITY_SOLD
------------ -------------------- -------------------
P003         Watch                                1
P001         Laptop                               1
P005         Bag                                  1
P002         Mobile                               1
P004         Headphones                           3
```

4. Products Requiring Promotion.

```
SQL> SELECT P.PRODUCT_ID,
  2         P.PRODUCT_NAME,
  3         P.PRICE,
  4         P.STOCK_QUANTITY,
  5         NVL(SUM(OD.QUANTITY), 0) AS QUANTITY_SOLD
  6  FROM PRODUCT P
  7  LEFT JOIN ORDER_DETAILS OD
  8  ON P.PRODUCT_ID = OD.PRODUCT_ID
  9  GROUP BY P.PRODUCT_ID,
 10           P.PRODUCT_NAME,
 11           P.PRICE,
 12           P.STOCK_QUANTITY
 13  HAVING NVL(SUM(OD.QUANTITY), 0) <= 1
 14     AND P.STOCK_QUANTITY > 0
 15  ORDER BY QUANTITY_SOLD, P.PRODUCT_NAME;

PRODUCT_ID   PRODUCT_NAME             PRICE STOCK_QUANTITY QUANTITY_SOLD
------------ -------------------- ---------- -------------- -------------
P005         Bag                        1500              8             1
P001         Laptop                     2500             10             1
P002         Mobile                     4500             20             1
P003         Watch                      3400             15             1
```

6. Perform Category-wise Sales Analysis

1. Total Sales Generated by Each Category

```
SQL> SELECT C.CATEGORY_NAME,
  2         SUM(OD.QUANTITY * P.PRICE) AS TOTAL_SALES
  3  FROM CATEGORY C
  4  JOIN PRODUCT P
  5  ON C.CATEGORY_ID = P.CATEGORY_ID
  6  JOIN ORDER_DETAILS OD
  7  ON P.PRODUCT_ID = OD.PRODUCT_ID
  8  GROUP BY C.CATEGORY_ID,
  9           C.CATEGORY_NAME
 10  ORDER BY TOTAL_SALES DESC;

CATEGORY_NAME
    TOTAL_SALES
--------------------------------------------------------------------------------
-------------------- -----------
Electronics
  14500
Wearables
   3400
Clothing
   1500
```

2. Number of Products Sold in Each Category

```
SQL> SELECT C.CATEGORY_NAME,
  2         SUM(OD.QUANTITY) AS TOTAL_PRODUCTS_SOLD
  3  FROM CATEGORY C
  4  JOIN PRODUCT P
  5  ON C.CATEGORY_ID = P.CATEGORY_ID
  6  JOIN ORDER_DETAILS OD
  7  ON P.PRODUCT_ID = OD.PRODUCT_ID
  8  GROUP BY C.CATEGORY_ID, C.CATEGORY_NAME
  9  ORDER BY TOTAL_PRODUCTS_SOLD DESC;

CATEGORY_NAME
    TOTAL_PRODUCTS_SOLD
----------------------------------------------------------------
-------------------- ------------------
Electronics
      5
Wearables
      1
Clothing
      1
```

3. Highest Revenue-Generating Category

```
SQL> SELECT C.CATEGORY_NAME,
  2         SUM(OD.QUANTITY * P.PRICE) AS TOTAL_REVENUE
  3  FROM CATEGORY C
  4  JOIN PRODUCT P
  5  ON C.CATEGORY_ID = P.CATEGORY_ID
  6  JOIN ORDER_DETAILS OD
  7  ON P.PRODUCT_ID = OD.PRODUCT_ID
  8  GROUP BY C.CATEGORY_ID, C.CATEGORY_NAME
  9  ORDER BY TOTAL_REVENUE DESC;

CATEGORY_NAME             TOTAL_REVENUE
------------------------- -------------
Electronics                     14500.00
Wearables                        3400.00
Clothing                         1500.00
```

4. Average Sales per Category

```
SQL> SELECT C.CATEGORY_NAME,
  2         AVG(P.PRICE * OD.QUANTITY) AS AVERAGE_SALES
  3  FROM CATEGORY C
  4  JOIN PRODUCT P
  5  ON C.CATEGORY_ID = P.CATEGORY_ID
  6  JOIN ORDER_DETAILS OD
  7  ON P.PRODUCT_ID = OD.PRODUCT_ID
  8  GROUP BY C.CATEGORY_ID, C.CATEGORY_NAME
  9  ORDER BY AVERAGE_SALES DESC;

CATEGORY_NAME             AVERAGE_SALES
------------------------- -------------
Electronics                      4833.33
Wearables                        3400.00
Clothing                         1500.00
```

7. Generate Business Analytics Reports

Report 1: Sales Performance Report

1. Total Sales

```
SQL> SELECT SUM(TOTAL_AMOUNT) AS TOTAL_SALES
  2  FROM ORDERS;

TOTAL_SALES
-----------
      19700
```

2. Number of Orders

```
SQL> SELECT COUNT(ORDER_ID) AS NUMBER_OF_ORDERS
  2  FROM ORDERS;

NUMBER_OF_ORDERS
----------------
               5
```

3. Average Order Value

```
SQL> SELECT AVG(TOTAL_AMOUNT) AS AVERAGE_ORDER_VALUE
  2  FROM ORDERS;

AVERAGE_ORDER_VALUE
-------------------
               3940
```

Report 2: Customer Analytics Report

- Customer Name,
- Number of Orders,
- Total Spending,
- Purchase Frequency

```
SQL> SELECT C.CUSTOMER_NAME,
  2         COUNT(O.ORDER_ID) AS NUMBER_OF_ORDERS,
  3         NVL(SUM(O.TOTAL_AMOUNT), 0) AS TOTAL_SPENDING,
  4         COUNT(O.ORDER_ID) AS PURCHASE_FREQUENCY
  5  FROM CUSTOMER C
  6  LEFT JOIN ORDERS O
  7  ON C.CUSTOMER_ID = O.CUSTOMER_ID
  8  GROUP BY C.CUSTOMER_ID, C.CUSTOMER_NAME
  9  ORDER BY TOTAL_SPENDING DESC;

CUSTOMER_NAME        NUMBER_OF_ORDERS TOTAL_SPENDING PURCHASE_FREQUENCY
-------------------- ---------------- -------------- ------------------
Divya                               1        7500.00                  1
Priya                               1        4500.00                  1
Karthik                             1        3400.00                  1
Arun                                1        2800.00                  1
Rahul                               1        1500.00                  1
```

Report 3: Product Performance Report

- Product Name
- Quantity Sold
- Revenue Generated

```
SQL> SELECT P.PRODUCT_NAME,
  2         SUM(OD.QUANTITY) AS QUANTITY_SOLD,
  3         SUM(OD.QUANTITY * P.PRICE) AS REVENUE_GENERATED
  4  FROM PRODUCT P
  5  JOIN ORDER_DETAILS OD
  6  ON P.PRODUCT_ID = OD.PRODUCT_ID
  7  GROUP BY P.PRODUCT_ID, P.PRODUCT_NAME
  8  ORDER BY REVENUE_GENERATED DESC;

PRODUCT_NAME         QUANTITY_SOLD REVENUE_GENERATED
-------------------- ------------- -----------------
Headphones                       3           7500.00
Mobile                           1           4500.00
Watch                            1           3400.00
Laptop                           1           2500.00
Bag                              1           1500.00
```

Report 4: Category Analysis Report

- Category Name
- Total Products Sold
- Total Revenue

```
SQL> SELECT C.CATEGORY_NAME,
  2         SUM(OD.QUANTITY) AS TOTAL_PRODUCTS_SOLD,
  3         SUM(OD.QUANTITY * P.PRICE) AS TOTAL_REVENUE
  4  FROM CATEGORY C
  5  JOIN PRODUCT P
  6  ON C.CATEGORY_ID = P.CATEGORY_ID
  7  JOIN ORDER_DETAILS OD
  8  ON P.PRODUCT_ID = OD.PRODUCT_ID
  9  GROUP BY C.CATEGORY_ID,
 10           C.CATEGORY_NAME
 11  ORDER BY TOTAL_REVENUE DESC;

CATEGORY_NAME             TOTAL_PRODUCTS_SOLD TOTAL_REVENUE
------------------------- ------------------- -------------
Electronics                                 5      14500.00
Wearables                                   1       3400.00
Clothing                                    1       1500.00
```